

QuickShow

Complete Technical Guide & Interview Preparation

Student	Akshath Adithya Jakkula
College	IIIT Allahabad
Branch	Electronics & Communication Engineering
Year	2nd Year (2023–Present)
Project Type	Full Stack Web Application
Live Link	https://quickshow-ak.vercel.app
GitHub	github.com/Adityajakkula7/QUICKSHOW

Tech Stack

Frontend	React.js, Tailwind CSS, React Router
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Auth	Clerk
Payment	Stripe
APIs	TMDB API, OpenStreetMap, Nominatim, OSRM
Maps	Leaflet, React-Leaflet
Deployment	Vercel (Frontend + Backend)

Table of Contents

- 1. Project Architecture Overview
- 2. Authentication — Clerk
- 3. CORS — Cross Origin Resource Sharing
- 4. REST API — Express.js Routes
- 5. MongoDB & Mongoose
- 6. TMDB API Integration
- 7. Stripe Payment Gateway
- 8. OpenStreetMap & Leaflet (Transport Feature)
- 9. React Frontend Architecture
- 10. Environment Variables & Security
- 11. Deployment — Vercel
- 12. Complete Interview Questions & Answers
- 13. Concepts Cheat Sheet

1 Project Architecture Overview

What you built:

QuickShow is a full-stack MERN application where users can browse real-time movies from TMDB API, select seats, book tickets with Stripe payment, and optionally book a cab to the theatre using OpenStreetMap. Each user's bookings are tied to their Clerk account.

System Flow:

Step	What Happens
User visits QuickShow	React frontend loads, Clerk checks if user is logged in
Browse Movies	Frontend calls backend → Backend calls TMDB API → Returns live movies
Select Movie	User sees movie detail with TMDB cast data + dynamic show timings
Choose Date & Time	Frontend generates next 7 days with 4 fixed timings (11:15, 2:30, 6:00, 9:00)
Select Seats	90 seats across 10 rows (A–J), max 5 seats per booking
Stripe Checkout	Backend creates Stripe session → User pays → Booking saved to MongoDB
My Bookings	User sees booking history fetched by their Clerk user ID
Book Cab	OpenStreetMap finds theatres within 20km → User selects → Route shown → OTP generated

2 Authentication Clerk

What is Clerk?

Clerk is a third-party authentication service that handles user sign-up, sign-in, session management, and user profiles out of the box. Instead of building auth from scratch (which involves JWT, password hashing, session tokens), Clerk provides a ready-made solution that is production-grade and secure.

Why Clerk over JWT?

Feature	JWT (Manual)	Clerk
Setup time	2-3 days	30 minutes
Password hashing	Manual (bcrypt)	Automatic
Session management	Manual	Automatic
Google OAuth	Complex setup	One click
Security	Depends on impl.	Enterprise grade
User dashboard	Build yourself	Provided

How Clerk works in QuickShow:

- ClerkProvider wraps the entire React app in main.jsx — makes auth state available everywhere
- useUser() hook returns the current logged-in user object with user.id, user.email etc.
- When user books a ticket, user.id is sent to backend and stored in MongoDB Booking model
- My Bookings page fetches bookings filtered by user.id — so each user sees only their bookings
- Login button uses Clerk's built-in SignInButton component — opens a modal with Google OAuth
- clerkMiddleware() on backend verifies Clerk tokens on protected routes

Code used:

```
import { useUser } from '@clerk/clerk-react'

const { user } = useUser() // gives current user

userId: user?.id || 'guest' // sent with booking
```

2 CORS Cross Origin Resource Sharing

What is CORS?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks requests from one domain (origin) to a different domain by default. For example, your frontend running on localhost:5173 cannot call your backend on localhost:3000 unless the backend explicitly allows it.

Why CORS is needed in QuickShow:

Frontend: <https://quickshow-ak.vercel.app> (different domain than backend)

Backend: <https://quickshow-sr.vercel.app> (different domain than frontend)

Without CORS, the browser would block all API calls from frontend to backend.

How CORS is set up in QuickShow:

```
import cors from 'cors'

app.use(cors()) // allows all origins (development)
```

In production, you should restrict CORS to specific origins:

```
app.use(cors({ origin: 'https://quickshow-ak.vercel.app' })))
```

CORS Headers explained:

Header	Purpose
Access-Control-Allow-Origin	Which domains can access the API
Access-Control-Allow-Methods	Which HTTP methods are allowed (GET, POST etc.)
Access-Control-Allow-Headers	Which headers can be sent in requests
Access-Control-Allow-Credentials	Whether cookies/auth headers are allowed

4. REST API - Express.js Routes

What is REST API?

REST (Representational State Transfer) is an architectural style for building APIs. It uses HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources. Each URL endpoint represents a resource.

All API Routes in QuickShow:

Method	Endpoint	Purpose
GET	/api/movies/now-playing	Fetch live movies from TMDB
GET	/api/movies/:id	Fetch specific movie with cast
GET	/api/shows/:movieId	Get show timings for a movie
POST	/api/shows/add	Admin: Add a new show
POST	/api/shows/generate	Auto-generate shows for 7 days
POST	/api/bookings/create-checkout-session	Create Stripe payment session
GET	/api/bookings/verify/:bookingId	Mark booking as paid after payment
GET	/api/bookings/user/:userId	Get all bookings for a user

Middleware used in server.js:

Middleware	Purpose
<code>express.json()</code>	Parse incoming JSON request bodies
<code>cors()</code>	Allow cross-origin requests from frontend
<code>clerkMiddleware()</code>	Verify Clerk auth tokens on requests
<code>dotenv/config</code>	Load environment variables from <code>.env</code> file

5 MongoDB & Mongoose

What is MongoDB?

MongoDB is a NoSQL database that stores data in JSON-like documents (called BSON) instead of tables and rows like SQL. It is flexible — documents in the same collection can have different fields. MongoDB Atlas is the cloud-hosted version used in QuickShow.

Models used in QuickShow:

Show Model (Show.js):

Field	Type	Purpose
movie	Number	TMDB movie ID
showDateTime	Date	Date and time of the show
showPrice	Number	Ticket price in rupees
occupiedSeats	Object	Map of seatId to userId
totalSeats	Number	Default 90 seats

Booking Model (Booking.js):

Field	Type	Purpose
user	String	Clerk user ID
movie	Number	TMDB movie ID
movieTitle	String	Movie name
moviePoster	String	Poster image URL
showDateTime	Date	Show date and time
bookedSeats	Array	List of seat IDs e.g. ['A1','A2']
amount	Number	Total amount paid
isPaid	Boolean	Payment status
stripeSessionId	String	Stripe session reference

SQL vs MongoDB:

SQL Term	MongoDB Term
Database	Database
Table	Collection
Row	Document
Column	Field
JOIN	Populate / Embed
SELECT	find()
INSERT	insertOne() / create()
UPDATE	updateOne() / findByIdAndUpdate()
DELETE	deleteOne() / findByIdAndDelete()

6 TMDB API Integration

What is TMDB?

TMDB (The Movie Database) is a free API that provides real-time movie data — titles, posters, cast, genres, ratings, release dates, and more. QuickShow uses TMDB to show real movies instead of dummy hardcoded data.

TMDB Endpoints used:

Endpoint	What it returns
/movie/now_playing	Movies currently playing in theatres
/movie/:id	Full movie details
/movie/:id?append_to_response=credits	Movie details + full cast list

Why call TMDB from backend not frontend?

- Security — API key is hidden in server .env, not exposed to browser
- Caching — Can cache results on server to reduce API calls
- No CORS issues — Server-to-server calls have no browser CORS restrictions
- Rate limiting — Easier to control from one place

Retry logic used:

Due to network instability, a `fetchWithRetry` function was implemented that retries the TMDB API call up to 3 times before returning an error. This handles `ECONNRESET` errors.

7 Stripe Payment Gateway

What is Stripe?

Stripe is a payment processing platform that handles online payments securely. It provides a hosted checkout page so you never handle raw card details — Stripe manages all PCI compliance.

Payment Flow in QuickShow:

1. **User selects seats:** Frontend collects seats, show time, movie info
2. **POST to backend:** `/api/bookings/create-checkout-session` called
3. **Save booking:** Booking saved to MongoDB with `isPaid: false`
4. **Create Stripe session:** `stripe.checkout.sessions.create()` called
5. **Redirect to Stripe:** User is redirected to Stripe's hosted payment page
6. **User pays:** Stripe handles card processing securely
7. **Success redirect:** Stripe redirects to `/my-bookings?success=true&bookingId=xxx`
8. **Verify payment:** Frontend calls `/api/bookings/verify/:id` → `isPaid` set to true
9. **Show bookings:** User sees their confirmed booking with Paid status

Key Stripe concepts:

Term	Meaning
Checkout Session	A hosted payment page created by Stripe
line_items	What the user is paying for (name, price, quantity)
unit_amount	Price in smallest currency unit (paise for INR)
success_url	Where Stripe redirects after successful payment
cancel_url	Where Stripe redirects if user cancels
Test mode	Use test cards (4242 4242 4242 4242) for testing

8. OpenStreetMap & Leaflet (Transport Feature)

What is OpenStreetMap?

OpenStreetMap (OSM) is a free, open-source mapping platform — like Google Maps but free. Leaflet is a JavaScript library for displaying interactive maps. Together they power QuickShow's cab booking transport feature.

APIs used in Transport feature:

API	Purpose	Cost
OpenStreetMap Tiles	Display the map background	Free
Nominatim API	Search for nearby theatres by query	Free
OSRM API	Calculate driving route between two points	Free
Browser Geolocation	Get user's current coordinates	Free (built-in)

Transport Feature Flow:

1. Browser asks for user location using `navigator.geolocation.getCurrentPosition()`
2. User's lat/lon coordinates are stored in state
3. Nominatim API searched for 'cinema' within 20km viewbox of user's location
4. Haversine formula used to filter results strictly within 20km radius
5. Theatre markers shown on Leaflet map — user clicks a theatre
6. OSRM API calculates driving route between user and theatre
7. Route drawn on map as a red polyline
8. Fare estimated: Base fare + (distance * per km rate) for Auto/Mini/Sedan
9. User clicks Book — random OTP, driver name, ETA generated and shown

Haversine Formula:

Used to calculate real-world distance between two lat/lon coordinates. This ensures theatres listed are truly within 20km and not just within the bounding box.

9. React Frontend Architecture

Pages in QuickShow:

Page	Route	What it does
Home	/	Hero section, Now Showing (TMDB), Trailers
Movies	/movies	All movies with language filter
MovieDetails	/movies/:id	Movie info, cast, date picker
SeatLayout	/movies/:id/:date	Seat selection + time picker
MyBookings	/my-bookings	User's booking history
Transport	/transport	Map, nearby theatres, cab booking
Releases	/releases	Upcoming movies
Favorites	/favorites	Saved movies

Key React concepts used:

Concept	Where used in QuickShow
useState	Store movies, bookings, seats, selected time etc.
useEffect	Fetch movies/shows on page load
useParams	Get movie ID and date from URL
useNavigate	Navigate between pages programmatically
useSearchParams	Read ?success=true&bookingId=xxx from URL
Props	Pass movie data to MovieCard, DateSelect components
Conditional rendering	Show Loading spinner while data fetches
Array.map()	Render list of movies, seats, bookings
async/await	All API calls use async/await pattern

10. Environment Variables & Security

What are Environment Variables?

Environment variables store sensitive configuration values like API keys, database URLs, and secrets outside of your source code. They are loaded at runtime from a `.env` file and never committed to GitHub.

Server .env variables:

Variable	Purpose
MONGODB_URI	MongoDB Atlas connection string
TMDB_API_KEY	TMDB API key for movie data
STRIPE_SECRET_KEY	Stripe secret key for payment processing
CLERK_SECRET_KEY	Clerk secret key for auth verification
CLIENT_URL	Frontend URL for Stripe redirect

Client .env variables (prefixed with VITE_):

Variable	Purpose
VITE_CLERK_PUBLISHABLE_KEY	Clerk public key for frontend auth
VITE_STRIPE_PUBLISHABLE_KEY	Stripe public key for frontend
VITE_GOOGLE_MAPS_KEY	Google Maps API key (if used)

Important: Always add `.env` to `.gitignore` so secrets are never pushed to GitHub!

11 Deployment Vercel

What is Vercel?

Vercel is a cloud platform for deploying frontend and backend applications. It provides automatic deployments from GitHub, custom domains, HTTPS, and a global CDN — all for free.

QuickShow Deployment:

Component	Platform	URL
Frontend (React)	Vercel	https://quickshow-ak.vercel.app
Backend (Node.js)	Vercel	https://quickshow-sr.vercel.app
Database (MongoDB)	MongoDB Atlas	Cloud hosted

Steps done for deployment:

1. Pushed code to GitHub repository
2. Imported GitHub repo into Vercel dashboard
3. Added all .env variables in Vercel's Environment Variables settings
4. Replaced all localhost:3000 with backend Vercel URL using Ctrl+Shift+H
5. Both frontend and backend auto-deploy on every GitHub push (CI/CD)

General Project Questions

Q: Tell me about your QuickShow project.

A: QuickShow is a full-stack movie ticket booking platform built with the MERN stack. It integrates TMDB API for real-time movies, Stripe for payments, Clerk for authentication, and has a unique transport feature using OpenStreetMap that lets users find nearby theatres and book cabs. It's deployed live on Vercel.

Q: Why did you choose MERN stack?

A: MERN (MongoDB, Express, React, Node) is the most popular JavaScript full-stack. Using JavaScript on both frontend and backend reduces context switching. MongoDB's flexible schema works well for movie and booking data that can vary in structure.

Q: What was the most challenging part?

A: The transport feature was most challenging — using browser geolocation, calling Nominatim API to find theatres, filtering by 20km using the Haversine formula, fetching routes from OSRM, and calculating fare estimates — all had to work together seamlessly.

Q: What is the unique feature of your project?

A: The cab booking transport feature is unique — no standard movie booking app has this. After booking tickets, users can find theatres within 20km, see the driving route on a map, choose between Auto/Mini/Sedan, get a fare estimate, and receive an OTP confirmation.

Q: How does a user book a ticket end to end?

A: User logs in with Clerk → browses TMDB movies → clicks movie → sees cast and details → picks a date from next 7 days → picks a time slot → selects up to 5 seats → clicks Proceed to Checkout → Stripe payment page → pays → redirected to My Bookings → booking shows as Paid → can then book a cab to the theatre.

Authentication Questions

Q: What is Clerk and why did you use it?

A: Clerk is a third-party authentication service. I used it because building auth from scratch with JWT, bcrypt, and session management takes days and has many security pitfalls. Clerk provides production-grade auth in 30 minutes with Google OAuth, email OTP, and user management dashboard included.

Q: How do you protect routes with Clerk?

A: On the backend, clerkMiddleware() is added to server.js which verifies the Clerk JWT token on every request. On the frontend, useUser() hook checks if user is logged in and redirects to login if not authenticated.

Q: How are bookings tied to users?

A: When a user books a ticket, their Clerk user.id is sent to the backend and stored in the Booking model's 'user' field. When fetching bookings, we filter by user.id so each user only sees their own bookings.

Q: What is the difference between authentication and authorization?

A: Authentication verifies who you are (login) — Clerk handles this. Authorization verifies what you can do (permissions) — for example, only admins can add shows. In QuickShow, we use Clerk for authentication and manually check user roles for authorization.

CORS & API Questions

Q: What is CORS and why is it needed?

A: CORS stands for Cross-Origin Resource Sharing. Browsers block requests from one domain to another by default for security. Since my frontend (quickshow-ak.vercel.app) and backend (quickshow-sr.vercel.app) are on different domains, I need CORS middleware on the backend to allow these cross-origin requests.

Q: What is the difference between GET and POST?

A: GET is used to retrieve data — no request body, data sent in URL. POST is used to send data to server — has a request body with JSON data. In QuickShow, GET /api/movies/now-playing fetches movies, POST /api/bookings/create-checkout-session creates a payment session.

Q: What is middleware in Express?

A: Middleware are functions that run between receiving a request and sending a response. In QuickShow: express.json() parses request body, cors() allows cross-origin requests, clerkMiddleware() verifies authentication. They run in order for every request.

Q: What is async/await and why is it used?

A: async/await is modern JavaScript syntax for handling asynchronous operations like API calls and database queries. Without it, you'd use .then() chains which are harder to read. Every API call in QuickShow uses async/await — e.g. const data = await fetch('/api/movies').

Q: How do you handle errors in your API?

A: Every route in QuickShow is wrapped in a try-catch block. If an error occurs (database error, API failure, etc.), the catch block returns a 500 status with success: false and the error message. The frontend checks data.success before using data.

Database Questions

Q: Why MongoDB over SQL for this project?

A: MongoDB's flexible document model suits movie data well — movies have different numbers of genres, cast members, and fields. SQL would require complex JOINS for the same data. Also, MongoDB Atlas provides free cloud hosting which is perfect for student projects.

Q: What is Mongoose and why use it?

A: Mongoose is an ODM (Object Document Mapper) for MongoDB. It adds schema validation, type checking, and helper methods on top of raw MongoDB. For example, Booking.find({ user: userId }) is cleaner than raw MongoDB queries.

Q: How is data structured in your Booking model?

A: The Booking model stores: user (Clerk ID), movie (TMDB ID), movieTitle, moviePoster, showDateTime, bookedSeats (array of seat IDs like ['A1','A2']), amount, isPaid (boolean), and stripeSessionId. This captures everything needed to show booking history.

Q: What is the difference between SQL and NoSQL?

A: SQL databases use structured tables with fixed schemas and support JOINS. NoSQL databases like MongoDB use flexible documents without fixed schemas. SQL is better for complex relationships, NoSQL is better for flexible/fast-changing data. QuickShow uses MongoDB because movie data structure varies and we need fast development.

Payment & Maps Questions

Q: How does Stripe payment work in your app?

A: When user clicks Proceed to Checkout: 1) Frontend sends booking details to backend. 2) Backend saves booking to MongoDB with isPaid:false. 3) Backend creates a Stripe Checkout Session with line items and redirect URLs. 4) Backend returns the Stripe URL. 5) Frontend redirects user to Stripe's hosted payment page. 6) After payment, Stripe redirects to /my-bookings?success=true&bookingId;=xxx. 7) Frontend calls verify endpoint which sets isPaid:true in MongoDB.

Q: Why don't you handle card details yourself?

A: Handling raw card data requires PCI DSS compliance — very complex and risky for small apps. Stripe's hosted checkout handles all of this securely. We never touch card numbers — Stripe processes everything and just tells us whether payment succeeded.

Q: How does the transport feature find nearby theatres?

A: First, browser's navigator.geolocation API gets user's lat/lon. Then, Nominatim API (OpenStreetMap's search service) is queried for 'cinema' within a 0.18 degree bounding box (approx 20km). Results are then filtered using the Haversine formula to ensure they're strictly within 20km of the user.

Q: What is the Haversine formula?

A: Haversine formula calculates the great-circle distance between two points on Earth given their latitude and longitude. It's more accurate than simple Euclidean distance because it accounts for Earth's curvature. Used in QuickShow to filter theatres that are truly within 20km radius.

Q: How is the driving route calculated?

A: OSRM (Open Source Routing Machine) is a free routing API that calculates driving routes. We send user's coordinates and theatre coordinates — OSRM returns the route as GeoJSON coordinates. Leaflet's Polyline component draws this route as a red line on the map.

React Questions

Q: What is the difference between useState and useEffect?

A: useState stores data that changes over time (movies list, selected seats, loading state). useEffect runs side effects like API calls when component mounts or when dependencies change. In QuickShow, useState stores movies and useEffect fetches them from the backend on page load.

Q: Why do you use useParams?

A: useParams extracts dynamic segments from the URL. For /movies/:id/:date, useParams gives { id: '1327819', date: '2026-03-24' }. This is used in MovieDetails and SeatLayout to know which movie and date to show.

Q: What is conditional rendering?

A: Showing different UI based on state. In QuickShow: show Loading spinner while movies are fetching, show 'No movies found' if empty array, show movies when loaded. Written as: {loading ? : movies.map(...)}

Q: How do you pass data between components?

A: Via props — parent passes data to child components. For example, FeaturedSection passes each movie object to MovieCard as a prop: . MovieCard receives it and displays the data.

12 Concepts Cheat Sheet

Key Terms to Know:

Term	Simple Definition
API	A way for two programs to talk to each other
REST API	API that uses HTTP methods (GET, POST, PUT, DELETE)
JSON	JavaScript Object Notation — data format like {key: value}
CORS	Browser security that blocks cross-domain requests by default
Middleware	Functions that run between request and response in Express
Authentication	Verifying who you are (login)
Authorization	Verifying what you can do (permissions)
JWT	JSON Web Token — encrypted token for authentication
Async/Await	Modern way to handle asynchronous JavaScript operations
useEffect	React hook to run side effects (API calls, subscriptions)
useState	React hook to store component-level data
Props	Data passed from parent to child component in React
MongoDB	NoSQL database storing data as JSON documents
Mongoose	ODM library that adds schema validation to MongoDB
Schema	Blueprint defining the structure of a database document
Environment Variables	Config values stored outside code for security
Deployment	Making your app accessible on the internet
Vercel	Cloud platform for deploying frontend and backend apps
CDN	Content Delivery Network — serves assets from nearest server
Haversine Formula	Calculates real distance between two geo coordinates
Geolocation API	Browser API to get user's current location
Nominatim	OpenStreetMap's geocoding/search service
OSRM	Free open-source routing engine for driving directions
Leaflet	JavaScript library for interactive maps
Stripe	Payment processing platform for online payments
Clerk	Third-party authentication service
TMDB	The Movie Database — free movie data API
Axios	HTTP client library for making API requests
dotenv	Library to load .env variables into Node.js process

Interview Tips

- **Start with the big picture:** Always explain the full flow first before diving into details. Say: 'QuickShow is a movie booking app where users can...' before talking about code.
- **Use numbers:** Interviewers love specifics. Say '90 seats across 10 rows' not 'many seats'. Say '20km radius' not 'nearby'. Say '4 show timings daily' not 'multiple timings'.
- **Know your unique feature:** The transport/cab booking feature is what sets you apart. Be ready to explain it in detail — how geolocation works, Haversine formula, OSRM routing.
- **Be honest about limitations:** If asked 'what would you improve?', say: 'I'd add real-time seat locking with Socket.io so two users can't book the same seat simultaneously.'
- **Show you understand why:** Don't just say what you did — say why. Not 'I used Clerk' but 'I used Clerk because building JWT auth from scratch has many security pitfalls.'
- **Live demo is powerful:** Open <https://quickshow-ak.vercel.app> during the interview. Show the full booking flow live. This impresses interviewers immediately.
- **Prepare for follow-ups:** Common follow-ups: 'How would you scale this?', 'How would you add real-time seat updates?', 'How would you add admin panel?'

All the best, Akshath! ■

You've built something genuinely impressive for a 2nd year student.

Live: <https://quickshow-ak.vercel.app>